

PRACTICAL-06

AIM: Implementation of chain matrix multiplication using dynamic programming.

PROGRAM:

```
#include <iostream>

#include <climits>

using namespace std;

int main() {

    int n;

    cout << "Enter number of matrices: ";

    cin >> n;

    int p[n + 1]; cout << "Enter dimensions array (" << n + 1
    << " values): "; for (int i = 0; i <= n; i++) { cin >> p[i];
    }

    int dp[n][n];

    // Cost is 0 when multiplying one
    matrix for (int i = 0; i < n; i++) {
        dp[i][i] = 0;
    }

    // chainLength is the number of matrices in a subchain for
    (int chainLength = 2; chainLength <= n; chainLength++) {
        for (int i = 0; i <= n - chainLength; i++) { int j = i +
        chainLength - 1;

Name:REVANTH BONTA , 5EN5, Lab Batch - A

        for (int k = i; k < j; k++) { int
            cost = dp[i][k] + dp[k + 1][j]
```

```
dp[i][j] = INT_MAX;

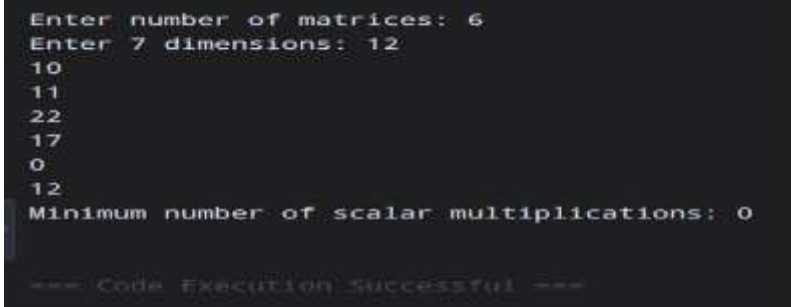
    + p[i] * p[k + 1] * p[j + 1];

    if (cost < dp[i][j]) {
        dp[i][j] = cost;
    }
}
}
}

cout << "Minimum number of scalar multiplications: "
    << dp[0][n - 1] << endl;

return 0;
}
```

OUTPUT:



```
Enter number of matrices: 6
Enter 7 dimensions: 12
10
11
22
17
0
12
Minimum number of scalar multiplications: 0

=== Code Execution Successful ===
```

CONCLUSION:

Matrix Chain Multiplication uses dynamic programming to find the most efficient order for multiplying matrices. It does not change the final matrix result; it only minimizes the number of scalar multiplications required. The C++ program computes this minimum cost efficiently in $O(n^3)$ time and uses $O(n^2)$ memory.